EXPERIMENT 10

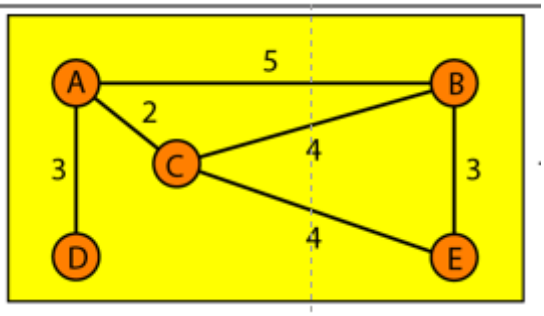
10. AIM: Write a Program to implement Dijkstra's algorithm to compute the Shortest path through a graph.

SOLUTION:

Algorithm: Dijkstra's Shortest Path

Step 1: Start.
Step 2: Initialize the graph with the given vertices and edge costs.
Step 3: Select the source node **A** and set its distance to **0**. Set the distances of all other nodes to ∞.
Step 4: Mark all nodes as **unvisited**.
Step 5: Select the unvisited node having the **smallest distance** and mark it as visited (permanent).
Step 6: Check all the unvisited neighboring nodes of the selected node.
Step 7: Calculate the new distance for each neighbor:
New distance = Current distance + Edge cost
If the new distance is smaller than the existing distance, **update the distance**.
Step 8: Repeat Steps 5–7 until all nodes are visited.
Step 9: Display the shortest distance from source node **A** to every other node.
Step 10: Stop.

Graph:



Source Code:

```
#include <stdio.h>

#define INF 9999
#define N 5

void dijkstra(int graph[N][N], int source)
{
    int distance[N];
    int visited[N];
    int i, j, min, next;

    // Initialize distances and visited array
    for (i = 0; i < N; i++)
    {
        distance[i] = graph[source][i];
        visited[i] = 0;
    }

    distance[source] = 0;
    visited[source] = 1;
```

```
    // Find shortest paths
    for (i = 1; i < N; i++)
    {
        min = INF;
        next = -1;

        // Find the vertex with minimum distance
        for (j = 0; j < N; j++)
        {
            if (!visited[j] && distance[j] < min)
            {
                min = distance[j];
                next = j;
            }
        }

        if (next == -1)
            break;

        visited[next] = 1;

        // Update distances
        for (j = 0; j < N; j++)
        {
            if (!visited[j] &&
                graph[next][j] != INF &&
                distance[next] + graph[next][j] < distance[j])
            {
                distance[j] = distance[next] + graph[next][j];
            }
        }
    }

    // Display shortest distances
    printf("\nShortest paths from A:\n");

    for (i = 0; i < N; i++)
    {
        printf("A -> %c = %d\n", 'A' + i, distance[i]);
    }
}

int main()
{
    int graph[N][N] =
    {
        // A    B    C    D    E
        { 0,   5,   2,   3, INF }, // A
        { 5,   0,   4, INF, 3 },   // B
        { 2,   4,   0, INF, 4 },   // C
        { 3, INF, INF, 0, INF },   // D
        { INF, 3,   4, INF, 0 }    // E
    };

    dijkstra(graph, 0);   // 0 represents A
```

```
    return 0;
}
```
Output: